Lab 6: Detecció (signatures, heurística) + YARA

Detecció d'amenaces

GEI | FIB | Curs 2026/2027

UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH
Facultat d'Informàtica de Barcelona



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH
Facultat d'Informàtica de Barcelona

Detecció d'amenaces

Assignatura amb 22 hores de laboratori corresponent a l'especialitat:

Tecnologies de la informació

Inclosa en el:

Grau en Enginyeria Informàtica

Sobre mi

- **MANEL RODERO**
- Administrador de Sistemes, Universitat Politècnica de Catalunya:
  - UTGCNTIC [2019 – actualitat]
  - inLab FIB [2012 – 2019]
  - LCFIB [1996 – 2012]
- Analista de Seguretat, esCERT-UPC [2013 – 2017]
- Enginyer Informàtic, Facultat d'Informàtica de Barcelona [1999]
- Contacte:
  - manel.rodero@upc.edu

Lab 6. Detecció (signatures, heurística) + YARA

Comprendre el funcionament intern dels sistemes de detecció

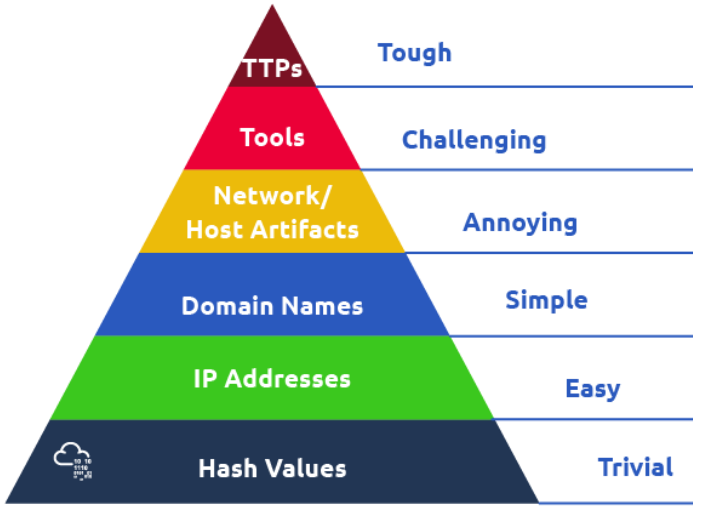
Els 4 enfocaments de la detecció

Enfocament	Com funciona	A la sessió d'avui
Signatures (hash)	Compara l'empremta **exacta** del fitxer; ràpida però fàcil d'eludir canviant únicament 1 byte	Exercici D4 (comparativa)
Signatures (patrons)	**Compara fragments de text/bytes dins el fitxer; més flexible**	**Regles YARA**
Heurística	Regles sobre **característiques sospitoses**, sense un patró concret (entropia, imports, seccions, etc.)	YARA + mòdul math
Comportament / IA	Observa què **fa** el procés en execució (processos, xarxa, fitxers, etc.)	EDR (altres sessions)

Combinació necessària: cap enfocament és suficient per si sol; els sistemes moderns combinen signatures, heurística i comportament.

Patró com a signatura

- Una **regla YARA** busca **fragments característics** *dins* del fitxer
  - Text, bytes (hex) o expressions regulars
  - Si el fitxer conté el patró → match, encara que la resta del fitxer canviï
- Un sol byte canviat → el **hash cau**, però el **patró sobreviu**
- El patró puja a la Piràmide del Dolor:
  - **Host Artifacts**: strings concrets d'una mostra
    - Una ruta, una comanda, una configuració, etc.
  - **Tools**: característiques de l'eina
    - Detecta tota la família (estructures del codi)
- Per esquivar-la, l'atacant ha de **modificar el codi**, no només el fitxer



Font: https://tryhackme.com/room/pyramidofpainax

Què és YARA i on s'utilitza?

- Eina de **codi obert** (VirusTotal) que permet **identificar** i **classificar** malware a partir de patrons:
  - Cadenes de text / seqüències hexadecimals / expressions regulars
- Escaneja fitxers, directoris (de forma recursiva) i també la memòria dels processos
- YARA és una eina transversal molt útil en:
  - **Anàlisi forense / Anàlisi de malware**: classificar mostres per famílies
  - **Threat Hunting**: buscar indicadors en molts equips
  - **Detecció**: signatures pròpies adaptades a l'entorn
- Integracions: Wazuh, VirusTotal, Velociraptor (DFIR)

Estructura d'una regla YARA

Nom: lletres, dígits, _ (no pot començar per dígit)

```
rule RuleName
{
    meta:
        author = "you"
        description = "what it detects"
        date = "2024-01-01"

    strings:
        $a = "malicious string"
        $b = { 4D 5A 90 00 }        // hex
        $c = /regex[0-9]{3}/i        // regex

    condition:
        $a and $b
}
```

Meta (opcional): informació sobre la regla, no afecta al match

Strings (opcional): què busquem

Condition: quan es dispara la regla

- Un fitxer `.yar` pot contenir **diverses regles**. Cadascuna s'avalua de manera independent
- Una regla **sense** strings és vàlida si la condició no en necessita
  - Ex. uint32(0) == ...

Executar YARA

- Yara s'executa amb:
  - yara [opcions] regles.yar OBJECTIU
- **Un sol objectiu per execució**
  - Per escanejar més d'un fitxer: fer servir un directori o un bucle for
- Opcions:
  - Cap: Mostra NomRegla fitxer per a cada match
  - -r: Escaneja un directori de forma **recursiva**
  - -s: Mostra els strings trobats i el seu **offset** (en hex)
  - -m: Mostra les metadades (meta)
  - -c: Mostra només el **nombre** de regles que fan match
  - -n: Mostra les regles que **no** fan match

```
# Instal·lació de YARA
sudo apt install yara -y
yara --version
```

Tipus d'strings

Tipus	Exemple	Notes
Text	"bash -i"	Escapaments: \\ \" \n \t \xNN
Hexadecimal	{ 7F 45 4C 46 }	Bytes exactes
	{ 4D 5A ?? 00 }	?? = qualsevol byte
	{ 4D 5A [2-4] 50 45 }	Salt de 2 a 4 bytes qualsevol
	{ 4D (5A | 5B) }	Alternativa
Regex	/\/dev\/(tcp|udp)\/[0-9.]+/	Sintaxi tipus Perl. Cal escapar la /

Dins d'un string de text, \ sempre inicia un escapament. "...[4\PZX54..." donarà un error: illegal escape sequence. Cal escriure "...[4\\PZX54...".

Expressions regulars: regex101: https://regex101.com/ | RegExr: https://regexr.com/

Modificadors d'strings

Modificador	Efecte	Exemple d'ús
nocase	Ignora majúscules / minúscules	XMRig, xmrig, XMRIG
fullword	Només coincidències de paraula sencera. L'string no pot anar precedit ni seguit de cap caràcter alfanumèric	"nc -e" no fa match dins "rsync -e"
wide	Busca cadenes en format UTF-16LE (c\0m\0d\0)	Típic de binaris Windows
base64	Busca l'string codificat en Base64	Payloads ofuscats

S'escriuen **després** de l'string: $s = "xmrig" nocase. Es poden combinar: "cmd.exe" wide nocase.

Condicions bàsiques

Expressió	Significat
$s1	$s1 apareix
$s1 and $s2 $s1 or $s2 not $s1	Lògica booleana (els parèntesis importen!)
any of them all of them	Almenys un dels strings Tots els strings
2 of them	Almenys 2 strings qualsevol
any of ($f*)	Almenys un dels strings que comencen per $f
$php at 0 and any of ($f*) and any of ($in*)	Combinació de grups

Agrupar per prefixos ($f1..$f4 = funcions, $in1..$in3 = entrades) fa la regla **llegible i ampliable**.

Condicions de posició, recompte i mida

Expressió	Significat
$s1 at 0	$s1 exactament a l'offset 0 (útil per a capçaleres de fitxers: **magic bytes**)
$s1 in (0..100)	$s1 dins dels primers 100 bytes
#s1 >= 3	$s1 apareix **3 o més** vegades
@s1[1]	Offset de la **1a** aparició de $s1
filesize < 1MB	Mida del fitxer (KB, MB)
uint16(0) == 0x5A4D	Llegeix 2 bytes a l'offset 0. MZ (PE de Windows)
uint32(0) == 0x464C457F	Llegeix 4 bytes a l'offset 0. ELF Linux

uint16/uint32 llegeixen en **little-endian**: el primer byte del fitxer és el **menys** significatiu. Per llegir en ordre "natural": uint32be(0) == 0x7F454C46

Mòduls: de la signatura a la heurística

- **Entropia de Shannon**
  - Mesura (de 0 a 8 bits/byte) com de "desordenats" són els bytes d'un fitxer:
    - Text pla ~ 4-5
    - Binari normal (/bin/ls) ~ 5,5-6,5
    - **Xifrat**, **comprimit**, **empaquetat** ~ 7,5-8
  - Una regla d'entropia **no busca cap patró** sinó una característica sospitosa (per això és **heurística**)

```
import "math"
import "hash"

rule Alta_Entropia {
    condition:
        filesize > 1KB and math.entropy(0, filesize) >= 7.5
}

rule Hash_Exacte {
    condition:
        hash.sha256(0, filesize) == "7836de28...a591c2"
}
```

  - El preu: un fitxer .zip o .jpeg legítims també fan match (**falsos positius**)
- Altres mòduls: pe (binaris de Windows: imports, seccions), elf, magic

Avaluar una regla: fals positiu i fals negatiu

- **Strings curts o genèrics** poden generar **falsos positius**
  - Ex.: "nc -e" també apareix dins de "rsync -e ssh"
  - Ajust: fullword, ancorar amb at/in, combinar strings (and, 2 of), filesize
- **Strings massa concrets** poden generar **falsos negatius**
  - Ex.: "bash -i" no detecta "sh -i"
  - Ajust: diverses variants amb any of, regex, nocase, base64
- **Compromís**: com més detecció, més soroll. Sempre cal validar la regla contra fitxers **legítims** abans de desplegar-la

	Fa match	**No fa match**
És maliciós	VP (vertader positiu) ✅	**FN** (fals negatiu): atac no detectat ❌
És legítim	**FP** (fals positiu): soroll per a l'analista ❌	VN (vertader negatiu) ✅

Un exemple real: WannaCry

Regla simplificada inspirada en deteccions públiques de WannaCry (2017) que es va propagar mitjançant EternalBlue (MS17-010), va xifrar fitxers amb AES-128 i va exigir el pagament en Bitcoin

```
rule WannaCry_Ransomware_Indicators
{
    meta:
        author = "dev.to reader"
        description = "Detects WannaCry-style dropper artifacts"
        reference = "https://www.cisa.gov/news-events/alerts/2017/05/12/wannacrypt-ransomware"
        date = "2024-11-01"
        tlp = "white"

    strings:
        // PE header
        $mz = { 4D 5A }

        // WannaCry kill switch domains (historical)
        $killswitch = "iuqerfsodp9ifjaposdfjhgosurijfaewrwergwea.com" nocase
        $killswitch2 = "ifferfsodp9ifjaposdfjhgosurijfaewrwergwea.com" nocase

        // Ransom note filename
        $note = "@Please_Read_Me@.txt" nocase

        // Bitcoin wallet strings used in the original campaign
        $btc1 = "13AM4VW2dhxYgXeQepoHkHSQuy6NgaEb94"
        $btc2 = "12t9YDPgwueZ9NyMgw519p7AA8isjr6SMw"

        // WannaCry helper tools
        $tor = "taskdl.exe" nocase
        $tor2 = "taskse.exe" nocase

        // Encryption-related imports
        $crypto = "CryptEncrypt" ascii
        $crypto_w = "CryptEncrypt" wide

    condition:
        $mz at 0 and
        (any of ($killswitch*) or
         $note or
         any of ($btc*) or
         (2 of ($tor*) and 1 of ($crypto*)))
}
```

$mz at 0 ancora el fitxer com a PE. Sense ell, un fitxer de text que inclogui l'adreça BTC coincidiria, un fals positiu de llibre de text

nocase en dominis detecta variacions de majúscules i minúscules que sobreviuen a l'ofuscació de cadenes

any of ($btc*) utilitza la coincidència de conjunts amb comodins, de manera que afegir més carteres més tard és barat.

$crypto i **$crypto_w** gestionen taules d'importació ASCII i UTF-16LE; les cadenes de l'API de Windows sovint apareixen en qualsevol de les dues formes

La condició és un OR de clústers independents. El programari maliciós muta; requerir tots els indicadors perdria variants. Requerir alguna combinació equilibra la recuperació i la precisió

Un exemple més real: WPSplinter (Oct'26)

One Parser, Two Bugs, Many Routers: Exploiting Broadcom WiFi Across eCos and Linux (Quentin Kaiser)



El router HGU y Livebox originales están afectados por una vulnerabilidad crítica en el wifi (BandaAncha.eu)

WPSplinter: Broadcom WPS EAP fragment reassembly vulnerability (Quentin Kaiser)

Broadcom 802.11 SDK Detection Toolkit (qkaiser @ GitHub)

Exercici dirigit D1: EICAR i primera regla YARA

- Crear o descarregar el fitxer eicar.com
- Crear el fitxer eicar_rule.yar amb la regla EICAR_Test_File
- Executar YARA: yara eicar_rule.yar eicar.com

```
alumne@ivysaur:~/lab6$ cat eicar.com
X5O!P%@AP[4\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*
alumne@ivysaur:~/lab6$ cat eicar_rule.yar
rule EICAR_Test_File
{
    meta:
        description = "Fitxer de test EICAR"
    strings:
        $eicar_string = "X5O!P%@AP[4\\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*"
    condition:
        $eicar_string
}
alumne@ivysaur:~/lab6$ yara eicar_rule.yar eicar.com
EICAR_Test_File eicar.com
alumne@ivysaur:~/lab6$
```

https://github.com/manelrodero/FIB-GEI-DA/

Exercicis dirigits D2 i D3

- Veure què ha trobat (-s i -m)
  - Executar YARA: yara -s -m eicar_rule.yar eicar.com
  - 0x0 = offset on comença la coincidència

```
alumne@ivysaur:~/lab6$ yara -s -m eicar_rule.yar eicar.com
EICAR_Test_File [description="Fitxer de test EICAR"] eicar.com
0x0:$eicar_string: X5O!P%@AP[4\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*
alumne@ivysaur:~/lab6$
```

- nocase
  - Crear el fitxer eicar_lower.com
  - Executar YARA: yara eicar_rule.yar eicar_lower.com (no hi haurà sortida)
  - Afegir "nocase" al final de la línia $eicar_string i escanejar el directori

```
alumne@ivysaur:~/lab6$ yara eicar_rule.yar .
EICAR_Test_File ./eicar.com
EICAR_Test_File ./eicar_lower.com
alumne@ivysaur:~/lab6$
```

Exercici dirigit D4: Hash vs Patró

- Copiar el fitxer eicar.com a eicar_mod.com
  - Modificar un byte (afegint-ne un caràcter)
- Calcular el SHA256 del fitxer eicar.com
- Calcular el SHA256 del fitxer eicar_mod.com
- Executar YARA: yara eicar_rule.yar eicar_mod.com

```
alumne@ivysaur:~/lab6$ sha256sum eicar.com
sha256sum eicar_mod.com
yara eicar_rule.yar eicar_mod.com
275a021bbfb6489e54d471899f7db9d1663fc695ec2fe2a2c4538aabf651fd0f  eicar.com
d01fa970d3e882abcd3df0673d2cb58a381205958512cb9d6e724e949fab23ac  eicar_mod.com
EICAR_Test_File eicar_mod.com
alumne@ivysaur:~/lab6$
```

Hash totalment diferent, la detecció per hash cau. La regla YARA continua detectant.

Exercici dirigit D5: Heurística/entropia amb math

- Crear el fitxer entropia.yar
- Crear un fitxer aleatori.bin (simulant un payload xifrat)
- Crear un fitxer comprimit legítim ls.gz de /bin/ls
- Executar YARA:
  - yara entropia.yar aleatori.bin
  - yara entropia.yar ls.gz

```
alumne@ivysaur:~/lab6$ file aleatori.bin
aleatori.bin: data
alumne@ivysaur:~/lab6$ file ls.gz
ls.gz: gzip compressed data, was "ls", last modified: Tue Aug 25 15:09:03 2026, from Unix, original size modulo 2^32 142
312
alumne@ivysaur:~/lab6$ for f in aleatori.bin ls.gz /bin/ls /etc/services; do yara entropia.yar $f; done
Alta_Entropia aleatori.bin
Alta_Entropia ls.gz
alumne@ivysaur:~/lab6$
```

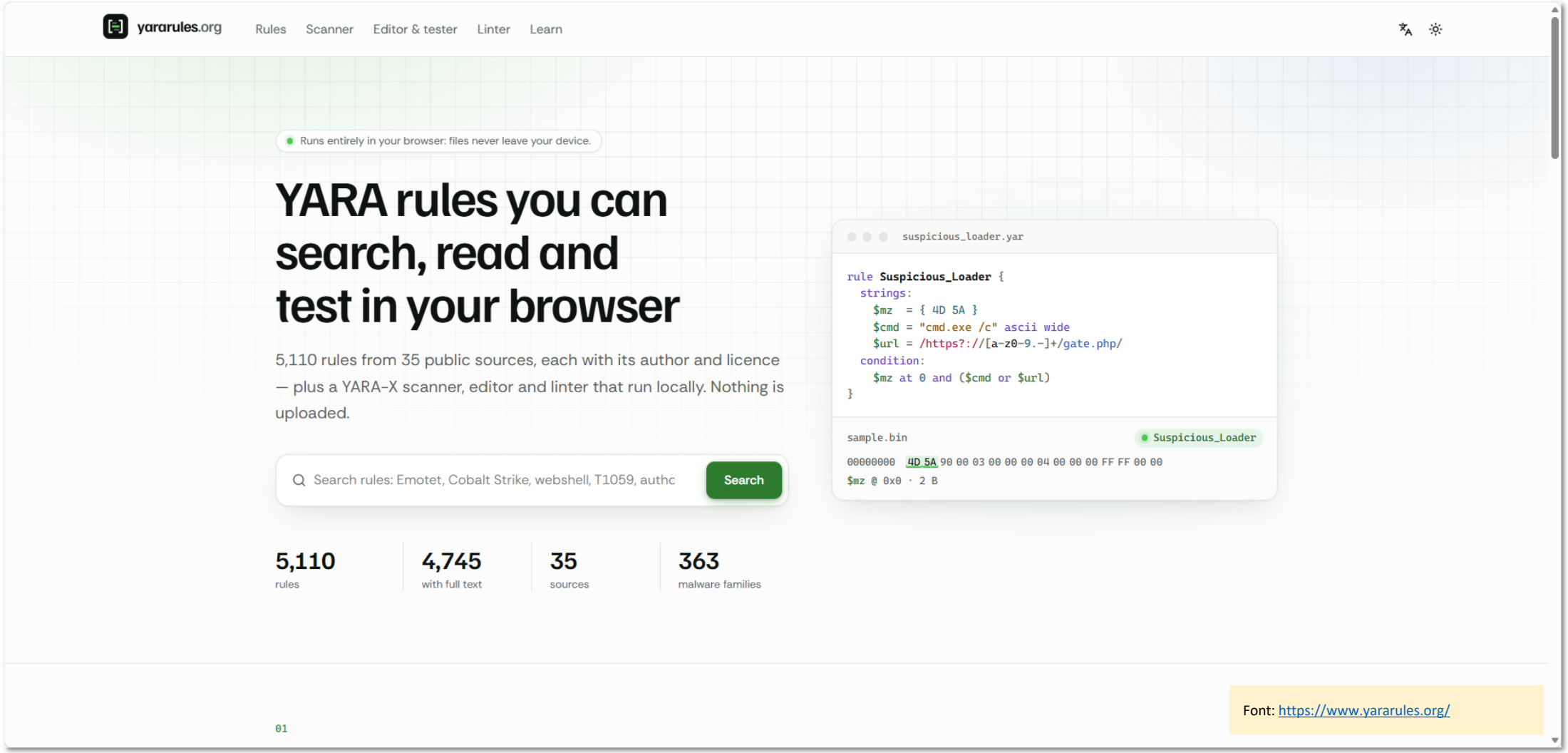
Es detecta el "payload xifrat" sense saber res del seu contingut, però ls.gz és un **fals positiu** (cal combinar)

Treball autònom en parelles



- El treball s'ha d'entregar en **1 únic fitxer PDF per grup** amb:
  - Contingut del fitxer *.yar
  - Comandes executades i resultat
  - Taules i/o fitxes de resultats
- El nom del fitxer ha de tenir aquest format:
  - `Lab6_grupXX_Cognom1_Cognom2.pdf`
- L'entrega es fa mitjançant el **Racó**
- Teniu fins a la **mitjanit** del següent dimarts o divendres per entregar

YaraRules.org



Font: https://www.yararules.org/

Detecció Reverse Shell i falsos positius (1 punt)

- Crear la regla Reverse_Shell_v1 al fitxer reverse_shell_v1.yar
  - La regla ha d'incloure 3 strings: "/dev/tcp/", "nc -e" i "bash -i"
  - La detecció s'ha de produir si al fitxer apareix **qualsevol** dels strings
- Executar YARA: yara reverse_shell_v1.yar mostres
- Crear la regla Reverse_Shell_v2 al fitxer reverse_shell_v2.yar
  - Ha de detectar **els 3 fitxers** sample_suspicious*.txt
  - **No** ha de fer match amb admin_notes.txt, rsync_backup.sh ni sample_clean.txt
- Executar YARA: yara reverse_shell_v2.yar mostres
- Omplir la taula de resultats de la pàgina següent

Primer cal executar un script per a preparar l'entorn

Pista: expressió regular per a "/dev/..." i el modificador adequat per a "nc -e"

Taula Reverse Shell (1 punt)

Fitxer	Match v1? (Si/No)	És maliciós? (Si/No)	Tipus v1 (VP/FP/VN/FN)	Match v2? (Si/No)	Tipus v2 (VP/FP/VN/FN)
sample_clean.txt					
sample_suspicious1.txt					
sample_suspicious2.txt					
sample_suspicious3.txt					
admin_notes.txt					
rsync_backup.sh					

Capçalera ELF i offsets (1 punt)

- Crear el fitxer elf_binary.yar amb dues regles:
  - ELF_Anywhere que busqui { 7F 45 4C 46 } a qualsevol lloc
  - ELF_Binary que faci el mateix però a l'offset 0
- Executar YARA:
  - yara -s elf_binary.yar /bin/ls
  - yara -s elf_binary.yar mostres/sample_clean.txt
  - yara -s elf_binary.yar mostres/fake_elf.txt
- Afegir una tercera regla ELF_Binary_uint32 **sense secció strings**
  - Cal fer servir uint32(0) == ...
  - *Quin valor hexadecimal heu escrit? Per què l'ordre dels bytes és aquest?*
  - Tornar a executar YARA
- *Omplir la taula de resultats de la pàgina següent*

Taula Capçalera ELF i offsets (1 punt)

Fitxer	ELF_Anywhere? (Si/No)	ELF_Binary? (Si/No)	ELF_Binary_uint32? (Si/No)	Offset trobat
/bin/ls				
sample_clean.txt				
fake_elf.txt				

WebShell PHP (1 punt)

- Crear la regla PHP_Webshell_Simple al fitxer php_webshell.yar
  - La regla ha de fer match amb **totes** les condicions indicades a continuació:
    - El fitxer comença (offset 0) amb <?php
    - El fitxer conté **alguna** funció d'execució: system(, exec(, passthru(, shell_exec(
    - El fitxer conté **alguna** entrada d'usuari: $_GET, $_POST, $_REQUEST
  - Cal utilitzar prefixos d'identificador: $f1..$f4, $in1..$in3
  - Cal utilitzar la coincidència de conjunt amb comodí: any of ($str*)
- Executar YARA:
  - yara php_webshell.yar mostres/webshell.php
  - yara php_webshell.yar mostres/index.php
- *Amb quina mostra fa match? Amb quina mostra **no** fa match?*
- *Per què no fa match si llegeix $_GET?*

Configuració de minador (1 punt)

- Crear la regla Cryptominer_Config al fitxer miner.yar
  - La regla ha de fer match amb **totes** les condicions indicades a continuació:
    - "stratum+tcp://" apareix **almenys 2 vegades**
    - apareix "xmrig" (sense distingir majúscules) **o** "donate-level"
  - Cal utilitzar l'operador de nombre d'ocurrències: #str
- Executar YARA i comprovar que fa match:
  - yara -s miner.yar mostres/config.json

- *Quantes vegades apareix "stratum+tcp://"?*
- *A quins offsets?*

Evasió per codificació (1 punt)

- El fitxer encoded.sh conté una Reverse Shell escrita en Base64
  - Les regles Reverse_Shell_v1 i Reverse_Shell_v2 **no** la detecten
- Crear la regla Reverse_Shell_Base64 al fitxer reverse_shell_b64.yar
  - Cal fer servir el modificador "base64" **únicament** sobre "/dev/tcp/"
- Executar YARA i comprovar que fa match
  - yara -s reverse_shell_b64.yar mostres/encoded.sh
- *Quina comanda està ofuscada al fitxer encoded.sh?*
- *A quin offset es troba la coincidència i quina cadena mostra YARA?*
- Executar: printf '/dev/tcp/' | base64.
- *Quin resultat dona? Apareix dins d'encoded.sh? Per què és diferent de la cadena mostrada per YARA?*

Reverse Shell Generator
Base64 Guru

Pista: Base64 treballa en blocs de 3 bytes